

AI Learning Buddy

Enter a Topic

merge sort

Choose Activity

Ask Anything



Generate

Merge Sort is an efficient, comparison-based, divide-and-conquer sorting algorithm. It's known for its guaranteed $O(n \log n)$ performance and stability.

How Merge Sort Works (The Divide-and-Conquer Strategy)

Merge Sort breaks down a list into several sub-lists until each sub-list consists of a single element (which is by definition sorted). Then, it repeatedly merges these sub-lists to produce new sorted sub-lists until there is only one sorted list remaining.

The process can be divided into three main steps:

1. **Divide:** Split the unsorted list into two halves (approximately equal size).
2. **Conquer (Recursively Sort):** Recursively apply Merge Sort to each half until you reach sub-lists of a single element. A list with one element is considered sorted.
3. **Combine (Merge):** Merge the sorted sub-lists back together to form a single sorted list. This is the crucial step where the actual sorting comparison happens.

The **Merge** Operation (The Heart of Merge Sort)

The core of Merge Sort is the `merge` function. It takes two *already sorted* sub-lists and combines them into a single sorted list.

Here's how the `merge` operation typically works:

1. Create a temporary array (or new list) to hold the merged result.
2. Use two pointers, one for each input sub-list, starting at their respective beginnings.
3. Compare the elements pointed to by the two pointers.
4. Copy the smaller element to the temporary array and advance its corresponding pointer.
5. Repeat step 4 until one of the sub-lists is exhausted.
6. Copy any remaining elements from the other (non-exhausted) sub-list to the temporary array.
These remaining elements are guaranteed to be larger than all elements already copied.
7. Finally, copy the elements from the temporary array back into the original array's position.

Example Walkthrough

Let's sort the array `[38, 27, 43, 3, 9, 82, 10]` using Merge Sort.

1. Divide Phase:

```
[38, 27, 43, 3, 9, 82, 10]
      / \
[38, 27, 43, 3]  [9, 82, 10]
    / \          / \
[38, 27] [43, 3] [9, 82] [10]
  / \   / \     / \
[38] [27] [43] [3] [9] [82] [10] (Base case: single-element lists are sorted)
```

2. Merge Phase (Working Back Up):

- Merge `[38]` and `[27]` → `[27, 38]`
- Merge `[43]` and `[3]` → `[3, 43]`
- Merge `[9]` and `[82]` → `[9, 82]`
- `[10]` remains as is (single element)

Now we have: `[27, 38] | [3, 43] | [9, 82] | [10]`

- Merge `[27, 38]` and `[3, 43]`
 - Compare 27 and 3 → take 3
 - Compare 27 and 43 → take 27
 - Compare 38 and 43 → take 38
 - Take remaining 43
 - Result: `[3, 27, 38, 43]`
- Merge `[9, 82]` and `[10]`
 - Compare 9 and 10 → take 9
 - Compare 82 and 10 → take 10
 - Take remaining 82
 - Result: `[9, 10, 82]`

Now we have: `[3, 27, 38, 43]` | `[9, 10, 82]`

- Finally, Merge `[3, 27, 38, 43]` and `[9, 10, 82]`
 - Compare 3 and 9 → take 3
 - Compare 27 and 9 → take 9
 - Compare 27 and 10 → take 10
 - Compare 27 and 82 → take 27
 - Compare 38 and 82 → take 38
 - Compare 43 and 82 → take 43
 - Take remaining 82
 - Result: `[3, 9, 10, 27, 38, 43, 82]` (The fully sorted list!)

Python Code Implementation

```
def merge_sort(arr):  
    # Base case: A list of 0 or 1 element is already sorted  
    if len(arr) <= 1:  
        return arr  
  
    # Divide: Find the middle point and split the list into two halves  
    mid = len(arr) // 2
```

```
left_half = arr[:mid]
right_half = arr[mid:]

# Conquer: Recursively sort each half
left_sorted = merge_sort(left_half)
right_sorted = merge_sort(right_half)

# Combine: Merge the two sorted halves
return merge(left_sorted, right_sorted)

def merge(left, right):
    result = []
    i = 0 # Pointer for the left array
    j = 0 # Pointer for the right array

    # Compare elements from both arrays and add the smaller one to the result
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add any remaining elements from the left array (if any)
    while i < len(left):
        result.append(left[i])
        i += 1

    # Add any remaining elements from the right array (if any)
    while j < len(right):
        result.append(right[j])
        j += 1

    return result

# Example Usage:
my_list = [38, 27, 43, 3, 9, 82, 10]
sorted_list = merge_sort(my_list)
print("Original List:", my_list)
print("Sorted List:", sorted_list)
```

```
my_list_2 = [12, 11, 13, 5, 6, 7]
sorted_list_2 = merge_sort(my_list_2)
print("Original List 2:", my_list_2)
print("Sorted List 2:", sorted_list_2)
```

Time and Space Complexity

- **Time Complexity:** $O(N \log N)$ in all cases (best, average, worst).
 - The `divide` step creates $\log N$ levels of recursion.
 - At each level, the `merge` operation processes N elements (across all sub-arrays), taking $O(N)$ time.
 - Therefore, the total time complexity is $N * \log N$.
- **Space Complexity:** $O(N)$ auxiliary space.
 - The `merge` function requires a temporary array of size N to store the merged results.
 - The recursion stack also takes $O(\log N)$ space, but the dominant factor is the auxiliary array for merging.

Characteristics of Merge Sort

- **Stable:** Yes, it maintains the relative order of equal elements.
- **Not In-Place:** Requires $O(N)$ extra space for the temporary arrays during merging.
- **Performance:** Excellent for large datasets and generally preferred when a stable sort is needed or when worst-case performance guarantees are critical.
- **Parallelizable:** Merge sort is well-suited for parallel processing.
- **External Sorting:** It's often used for external sorting, where the data to be sorted is too large to fit into RAM.

When to Use Merge Sort

- When a stable sort is required.
- When $O(N \log N)$ worst-case time complexity is a critical guarantee.

- For sorting linked lists efficiently, as merging linked lists doesn't require extra space beyond new pointers.
- For external sorting of very large files.